# Cahier des charges — Osaker_Kernel

Oct 21, 2026 · @Rami

## 1. Contexte et objectifs

Le projet Osaker_Kernel consiste à développer from scratch un noyau de système d'exploitation pour l'IoT, destiné aux processeurs applicatifs ARM 64 bits (AArch64, avec MMU) et écrit conjointement en C et en Rust. Il s'agit d'un projet personnel à finalité pédagogique, mené en parallèle d'une alternance en cybersécurité, dans le cadre d'une réorientation vers le développement système bas niveau orienté sécurité.

Le noyau vise des usages IoT et robotiques : piloter capteurs et actionneurs, communiquer sur le réseau et garantir des temps de réponse maîtrisés. Les orientations sont hiérarchisées ainsi :

| Priorité | Orientation | Traduction dans le projet |
|---|---|---|
| 1 | Temps réel | Ordonnanceur préemptif à priorités fixes, mutex à héritage de priorité, boucles de contrôle déportées sur un cœur temps réel compagnon |
| 2 | Sécurité | Isolation mémoire, mitigations matérielles ARM, étude de TrustZone et du secure boot |
| 3 | Connectivité | Ethernet et pile réseau minimale |
| 4 | Basse consommation | Mise en veille du CPU au repos, puis gestion d'énergie avancée |

Le cahier des charges privilégie l'apprentissage à la performance : la cohabitation des deux langages est imposée même lorsqu'elle n'est pas le choix technique optimal.

**Objectifs**

1. **Maîtriser le C freestanding** : programmation sans bibliothèque standard, gestion manuelle de la mémoire, interaction directe avec le matériel.
2. **Maîtriser le Rust `no_std`** : développement noyau sans runtime, usage encadré de `unsafe`, allocateur global personnalisé.
3. **Maîtriser l'interopérabilité C/Rust** dans les deux sens, selon le modèle de Rust-for-Linux.
4. **Comprendre le fonctionnement interne d'un OS sur ARM** : niveaux d'exception, MMU, contrôleur d'interruptions, ordonnancement temps réel.

5. **Comprendre les SoC hétérogènes** (cœur applicatif + cœur temps réel), modèle de la robotique et de l'IoT industriel.
6. **Constituer un projet de portfolio** démontrable pour des postes de développement système, embarqué ou driver.
7. **Fournir un terrain d'étude sécurité** : implémentation de mitigations, puis analyse de vulnérabilités sur un code maîtrisé de bout en bout.

Le C++ pourra être intégré ultérieurement, en espace utilisateur uniquement.

## 2. Périmètre et plateformes cibles

Le noyau est développé sur deux plateformes en parallèle : la machine virtuelle QEMU `virt`, qui ne nécessite aucun matériel, puis une carte de développement réelle. Tant que le noyau fonctionne sur les deux, aucune dépendance à un matériel particulier ne s'est glissée dans le cœur.

| Plateforme | Rôle | Jalons |
|---|---|---|
| QEMU `virt` (AArch64) | Plateforme de développement et de CI, permanente | Dès M0 |
| STM32MP257F-DK (2× Cortex-A35 + Cortex-M33) | Cible matérielle principale : temps réel, sécurité, robotique | À partir de M4 |
| BeaglePlay | Cible de repli complète (AArch64 avec cœurs temps réel) | Si la carte principale est indisponible |
| Raspberry Pi 4 | Cible de repli pour valider le premier portage sur matériel | Au besoin |

| Inclus | Exclu |
|---|---|
| Architecture AArch64, noyau en niveau d'exception EL1 | Microcontrôleurs sans MMU comme cible du noyau |
| Démarrage direct par QEMU, puis via TF-A et U-Boot sur carte | Développement d'un bootloader complet (étudié en phase sécurité) |
| Un seul cœur applicatif dans un premier temps | Multiprocesseur symétrique (SMP), prévu en évolution |
| Mémoire virtuelle, heap noyau, espaces d'adressage isolés | Swap, mémoire partagée entre processus |

| Inclus | Exclu |
|---|---|
| Ordonnancement temps réel à priorités fixes | Temps réel certifié (ISO 26262, IEC 61508) |
| Espace utilisateur, appels système, chargeur ELF | Compatibilité binaire Linux ou POSIX complète |
| VFS, tmpfs, initrd | Systèmes de fichiers sur disque en écriture |
| Drivers : UART, timer, contrôleur d'interruptions, GPIO, PWM, CAN-FD, Ethernet | USB, Wi-Fi, Bluetooth, affichage, caméra |
| Firmware bare-metal du Cortex-M33 et communication inter-cœurs | Interface graphique |

## 3. Exigences fonctionnelles

Chaque exigence est identifiée (EF-xx), rattachée à un sous-système et à son langage d'implémentation. La priorité suit la méthode MoSCoW : M (obligatoire), S (souhaitable), C (optionnel).

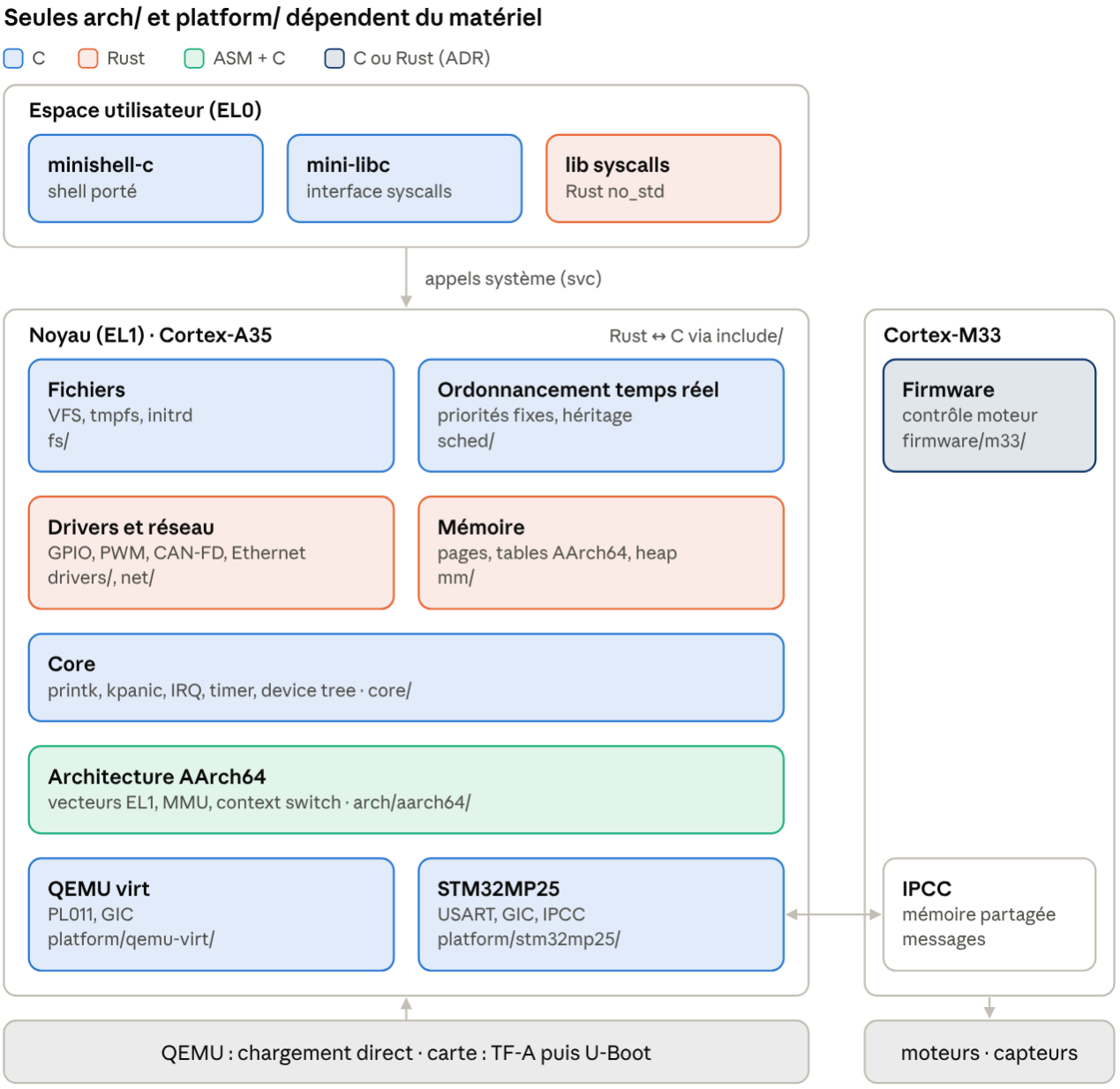
| ID | Sous-système | Exigence | Langage | Priorité |
|---|---|---|---|---|
| EF-01 | Boot | Le noyau démarre en EL1, reçoit l'adresse du device tree et s'exécute en higher-half | ASM + C | M |
| EF-02 | Core | Le noyau affiche des messages formatés ( `printk` ) sur l'UART de la plateforme (PL011 sous QEMU, USART sur STM32MP25) | C | M |
| EF-03 | Core | Toute erreur fatale passe par un `kpanic()` unique qui affiche les registres, ESR et FAR, et la pile, puis arrête le système | C | M |
| EF-04 | Plateforme | Le device tree (FDT) est parsé au boot pour découvrir la RAM, l'UART, le contrôleur d'interruptions et le timer | C | M |

| ID | Sous-système | Exigence | Langage | Priorité |
|---|---|---|---|---|
| EF-05 | Arch | Table de vecteurs EL1 installée (VBAR_EL1) ; les exceptions synchrones sont décodées via ESR_EL1 | ASM + C | M |
| EF-06 | Arch | Les interruptions matérielles sont gérées via le GIC (v2 ou v3 selon la plateforme) | C | M |
| EF-07 | Core | Le timer générique ARM fournit le tick système et des échéances ponctuelles | C | M |
| EF-08 | Mémoire | Un allocateur de pages physiques gère la RAM déclarée par le device tree | Rust | M |
| EF-09 | Mémoire | Le noyau gère ses tables de pages AArch64 (granule 4 Kio, TTBR0 / TTBR1), avec attributs mémoire normale et device | Rust | M |
| EF-10 | Mémoire | Un heap noyau expose `kmalloc` / `kfree` au C et sert d'allocateur global Rust | Rust | M |
| EF-11 | Ordonnancement | Création et destruction de threads noyau avec changement de contexte | ASM + C | M |
| EF-12 | Temps réel | Ordonnanceur préemptif à priorités fixes, round-robin entre threads de même priorité | C | M |
| EF-13 | Temps réel | Mutex à héritage de priorité, spinlocks et sémaphores | C | M |
| EF-14 | Temps réel | Mode tickless et instrumentation des latences d'interruption et de commutation | C | S |
| EF-15 | Drivers | Un trait `Driver` en Rust encapsule l'API C ; les drivers sont instanciés depuis le device tree | Rust | M |
| EF-16 | Drivers | GPIO, PWM et CAN-FD sur la carte | Rust | S |

| ID | Sous-système | Exigence | Langage | Priorité |
|---|---|---|---|---|
| EF-17 | Inter-cœurs | Le noyau charge et démarre le firmware du Cortex-M33, puis échange des messages via mémoire partagée et mailbox matérielle (IPCC) | C | S |
| EF-18 | Firmware M33 | Firmware bare-metal temps réel : boucle de contrôle à fréquence fixe, PWM, encodeurs | C ou Rust (ADR) | S |
| EF-19 | Userland | Les processus s'exécutent en EL0 dans un espace d'adressage isolé | C | M |
| EF-20 | Userland | Les appels système passent par `svc` avec validation des arguments | C | M |
| EF-21 | Userland | Un chargeur ELF64 charge et lance un exécutable statique | C | M |
| EF-22 | Fichiers | Une couche VFS unifie l'accès aux fichiers ; tmpfs et initrd sont montés au démarrage | C | M |
| EF-23 | Processus | `spawn`, `exit`, `wait`, puis `fork` / `exec` ; tubes et redirections ( `pipe`, `dup2` ) | C | S |
| EF-24 | Userland | Une mini-libc C et une bibliothèque Rust `no_std` exposent les appels système | C + Rust | M |
| EF-25 | Userland | Le shell `minishell-c` existant est porté et s'exécute sur le noyau | C | S |
| EF-26 | Réseau | Driver Ethernet (virtio-net sous QEMU, contrôleur de la carte) et pile UDP/IP minimale | Rust | S |
| EF-27 | Énergie | Mise en veille du CPU au repos ( `wfi` ) | C | S |
| EF-28 | Userland | Un programme utilisateur écrit en C++ | C++ | C |

# 4. Architecture technique

Le noyau est monolithique, découpé en sous-systèmes aux interfaces nettes. Le C porte le socle matériel, le cœur et l'ordonnancement temps réel ; le Rust prend en charge la mémoire, les drivers et le réseau. Les boucles de contrôle temps réel tournent sur le Cortex-M33, hors du noyau, qui communique avec lui par messages.




architecture en couches · répartition C / Rust · cœur temps réel compagnon

Seules les couches `arch/aarch64/` et `platform/` connaissent le matériel : changer de carte revient à écrire un nouveau répertoire `platform/` et les drivers spécifiques au SoC, sans toucher au cœur. Les crates Rust n'appellent le noyau qu'à travers les headers de `include/`, selon les règles de la section 5.

# 5. Règles d'interopérabilité FFI

Aucun compilateur ne vérifie la cohérence entre le code C et le code Rust : ces quatre règles sont donc obligatoires et vérifiées en revue de code.

| ID | Règle | Risque couvert |
|---|---|---|
| FFI-01 | Les headers C de `include/` sont la source de vérité. Les bindings Rust sont générés par `bindgen`, les headers des fonctions Rust exportées par `cbindgen`, à chaque build. Aucune déclaration FFI n'est écrite à la main. | Signatures ou layouts de structures désynchronisés |
| FFI-02 | Les crates Rust sont compilées avec `panic = "abort"` ; le panic handler appelle `kpanic()`. Aucune panic ne traverse la frontière. | Unwinding à travers des frames C, état incohérent |
| FFI-03 | Seuls des types de layout C traversent la frontière : `#[repr(C)]`, entiers à taille fixe, pointeurs bruts. Les données reçues du C sont validées avant conversion en types Rust sûrs. | Violation des invariants Rust (références nulles, aliasing, `bool` ou `enum` invalides) |
| FFI-04 | Chaque fonction exportée documente l'ownership : qui alloue, qui libère, durée de vie des pointeurs. | Double free, fuite, use-after-free |

Tout bloc `unsafe` est précédé d'un commentaire `// SAFETY:` justifiant les invariants respectés, et la couche `unsafe` reste confinée aux modules d'interface.

# 6. Exigences non fonctionnelles

La règle de portabilité ENF-01 est la plus importante : c'est elle qui garantit qu'aucun travail n'est perdu si la carte change. Les exigences de qualité s'appliquent dès M0, les mitigations de sécurité au fil des jalons.

| ID | Domaine | Exigence | Jalon |
|---|---|---|---|
| ENF-01 | Portabilité | Aucune adresse matérielle ni aucun détail de SoC en dehors de `arch/` et `platform/` ; vérifié à chaque revue de code | M0 |

| ID | Domaine | Exigence | Jalon |
|---|---|---|---|
| ENF-02 | Portabilité | Le matériel est découvert via le device tree ; QEMU `virt` reste une cible permanente de la CI | M1 |
| ENF-03 | Qualité C | Compilation sans avertissement avec `-Wall -Wextra -Werror` ; analyse statique par `clang-tidy` | M0 |
| ENF-04 | Qualité Rust | Aucun avertissement `clippy` ; `rustfmt` appliqué | M0 |
| ENF-05 | Tests | Tests unitaires hôtes pour la logique pure (allocateurs, ordonnanceur, structures de données) ; tests d'intégration exécutés dans QEMU | M2 |
| ENF-06 | CI | Chaque commit sur la branche principale est compilé et démarré dans QEMU par GitHub Actions | M0 |
| ENF-07 | Debug | GDB via le stub QEMU (avec GEF à partir de M1) ; GDB via OpenOCD et le ST-LINK intégré sur carte | M0 |
| ENF-08 | Temps réel | Latences d'interruption et de commutation mesurées sur carte et consignées à chaque jalon, QEMU n'étant pas représentatif des timings | M4 |
| ENF-09 | Sécurité | W^X : aucune page noyau n'est à la fois inscriptible et exécutable | M2 |
| ENF-10 | Sécurité | Bits PXN / UXN : le noyau ne peut pas exécuter de code utilisateur, ni l'utilisateur du code noyau | M5 |
| ENF-11 | Sécurité | Accès noyau à la mémoire utilisateur uniquement via des fonctions de copie dédiées ; PAN activé si le CPU le supporte (absent du Cortex-A35, ARMv8.0), sinon émulé par bascule de TTBR0 | M5 |
| ENF-12 | Sécurité | Validation systématique des pointeurs et longueurs reçus par les appels système | M5 |
| ENF-13 | Sécurité | Page de garde sous chaque pile noyau (M3) ; stack canaries sur le code C (M5) | M3, M5 |

| ID | Domaine | Exigence | Jalon |
|---|---|---|---|
| ENF-14 | Sécurité | KASLR (randomisation de l'adresse de base du noyau) | Post-M10 |
| ENF-15 | Sécurité | Étude de TrustZone et de la chaîne de secure boot TF-A sur STM32MP25 | Post-M10 |
| ENF-16 | Documentation | README de build, document d'architecture tenu à jour, décisions structurantes consignées en ADR | Continu |
| ENF-17 | Gestion de version | Git, branches par fonctionnalité, commits au format Conventional Commits, tag à chaque jalon | Continu |
| ENF-18 | Évolutivité | Code prêt pour le SMP : spinlocks à opérations atomiques, données par cœur dans une structure per-CPU (registre `TPIDR_EL1`), une file d'ordonnancement par cœur | M3 |
| ENF-19 | Évolutivité | Espaces d'adressage représentés par régions, avec comptage de références sur les pages physiques (prérequis de la mémoire partagée et du swap) | M2 |
| ENF-20 | Évolutivité | Noyau recompilable en code relogeable (PIE) avec auto-relocation au boot, prérequis de KASLR ; aucune adresse de chargement figée hors du linker script | M2 |
| ENF-21 | Sûreté | Démarche inspirée des normes de sûreté (IEC 61508), sans certification : sous-ensemble MISRA C vérifié par `cppcheck`, matrice de traçabilité exigences → code → tests, couverture mesurée (`llvm-cov`), temps d'exécution mesurés sur carte pour le firmware M33 et les chemins critiques du noyau | M4, puis continu |

## 7. Environnement de développement et chaîne de build

Un Makefile unique compile le C, appelle `cargo` pour les crates Rust et lie le tout en un ELF AArch64 higher-half, lancé directement par QEMU ou chargé par U-Boot sur la carte.

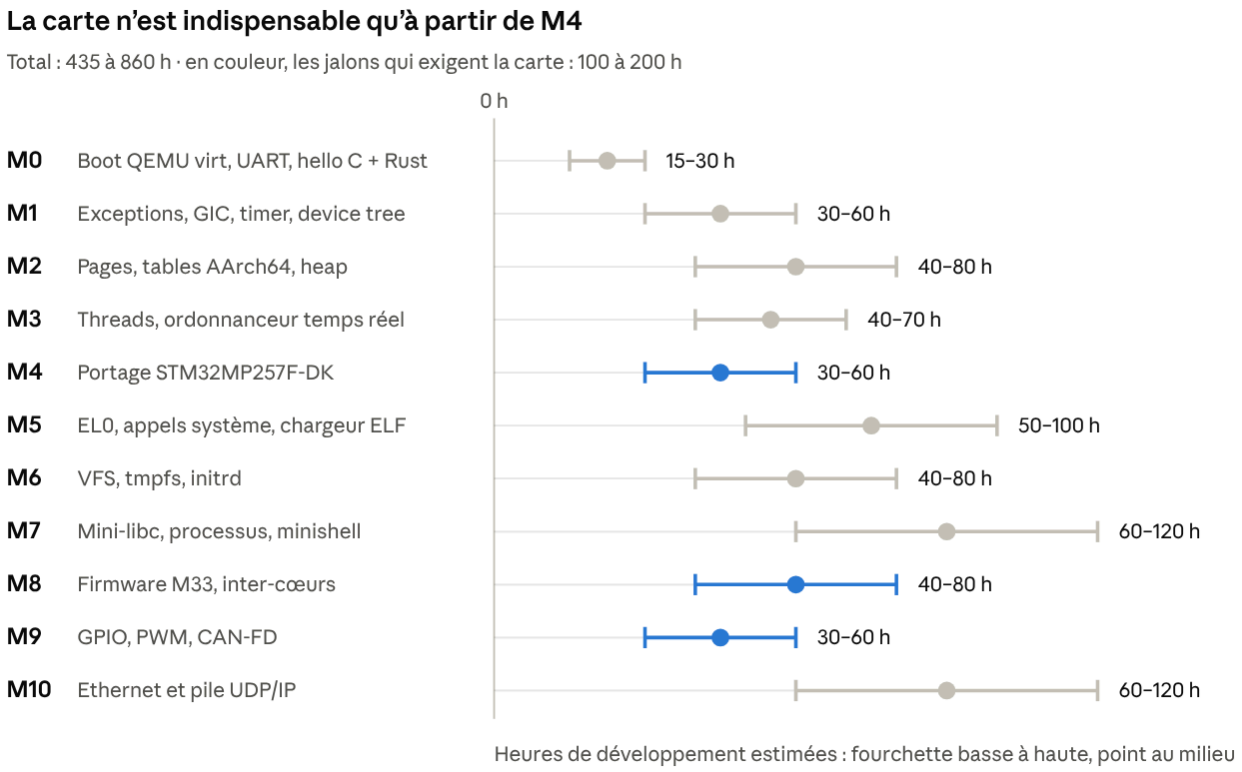
| Élément | Choix | Remarque |
|---|---|---|
| Compilateur C | clang `--target=aarch64-none-elf`, freestanding | `-ffreestanding -mgeneral-regs-only -fno-pic -mstrict-align` (accès alignés obligatoires tant que la MMU est coupée) |
| Rust | toolchain épinglée via `rust-toolchain.toml` | Cible `aarch64-unknown-none-softfloat` (pas de registres FP/SIMD dans le noyau), crates en `staticlib`, `no_std` |
| Assembleur | Assembleur intégré de clang (syntaxe GNU) | Boot, vecteurs d'exception, context switch |
| Liaison | `ld.lld` + `linker.ld` | Sections alignées sur 4 Kio pour appliquer W^X |
| Bindings | `bindgen`, `cbindgen` | Exécutés par le Makefile à chaque build (FFI-01) |
| Émulation | `qemu-system-aarch64 -M virt -cpu cortex-a53` | `-serial stdio` ; `-cpu max` pour tester les extensions de sécurité récentes (PAN) |
| Boot sur carte | TF-A puis U-Boot depuis microSD | U-Boot charge le noyau comme un binaire ; chaîne de boot non modifiée avant la phase sécurité |
| Debug | GDB + GEF (fork bata24) ; OpenOCD + ST-LINK sur carte | `tools/gdbinit` propre au projet, chargé par `make debug` |
| CI | GitHub Actions | Build + démarrage QEMU avec délai maximal |
| Matériel | STM32MP257F-DK, chargeur USB-C 5 V / 3 A, carte microSD | Nécessaire à partir de M4 seulement |

### Arborescence cible

```
kernel/
├── Makefile
├── linker.ld
├── arch/aarch64/       # ASM + C : boot, vecteurs EL1, MMU, context switch
├── platform/
│   ├── qemu-virt/      # C : PL011, GIC, carte mémoire
│   └── stm32mp25/      # C : USART, horloges, IPCC
├── core/               # C : printk, panic, irq, timer, device tree
├── include/            # headers C = contrat FFI
├── mm/                 # crate Rust : pages, tables, heap
├── sched/              # C : threads, ordonnanceur temps réel,
synchronisation
├── drivers/            # crate Rust : trait Driver + drivers
├── net/                # crate Rust : pile UDP/IP
├── fs/                 # C : vfs, tmpfs, initrd
├── rust/kernel-sys/    # bindings générés par bindgen
├── user/               # libc C, programmes C/Rust
├── firmware/m33/       # firmware temps réel du Cortex-M33
├── docs/               # architecture, ADR
└── tools/              # scripts QEMU, GDB, CI
```

## 8. Planning, jalons et estimation de charge

Le projet se découpe en onze jalons séquentiels, chacun dépendant du précédent. Les quatre premiers se font entièrement sur QEMU : la carte ne devient nécessaire qu'au portage (M4), puis pour le cœur temps réel et les périphériques (M8, M9).




estimations issues de l'échange · debug et apprentissage inclus

Le socle M0 à M4 forme à lui seul un noyau complet sur carte réelle, soit 4 à 7 mois à 10 h par semaine. Le projet entier représente 10 à 20 mois à ce rythme, le double à 5 h par semaine. Le durcissement sécurité (30 à 60 h) vient en plus, et le surcoût lié aux deux langages (20-30 %) est concentré sur M0 et M2.

- [ ] Fixer la date de démarrage et le rythme hebdomadaire cible
- [ ] Commander la STM32MP257F-DK, ou une carte de repli, avant la fin de M3

## 9. Critères de recette

Un jalon est validé lorsque tous ses critères sont vérifiés par un test automatisé (ou, à défaut, par une procédure manuelle documentée), que la CI est verte sur QEMU `virt` et que le tag Git du jalon est posé.

| Jalon | Critères de recette | Exigences |
|---|---|---|
| M0 | L'image démarre sous QEMU `virt` ; une ligne écrite par le C puis une ligne écrite par une fonction Rust appelée depuis le C apparaissent sur l'UART ; GDB s'arrête sur le point d'entrée | EF-01, EF-02, FFI-01 à FFI-03 |

| Jalon | Critères de recette | Exigences |
|---|---|---|
| M1 | Un accès mémoire invalide et une instruction non définie provoqués volontairement sont interceptés et décodés (ESR, FAR) ; le tick progresse ; le contenu du device tree est affiché | EF-03 à EF-07, ENF-02 |
| M2 | 10 000 cycles allocation / libération sans fuite ; une écriture sur une page de code déclenche une faute de permission (W^X) | EF-08 à EF-10, ENF-09, ENF-19, ENF-20 |
| M3 | Le thread le plus prioritaire préempte toujours les autres ; une inversion de priorité provoquée volontairement est résolue par l'héritage ; un compteur partagé protégé par mutex est exact après 1 000 000 d'incréments | EF-11 à EF-13, ENF-13, ENF-18 |
| M4 | Le noyau démarre sur la STM32MP257F-DK via U-Boot, écrit sur l'UART et reçoit le tick, avec le même code cœur que sous QEMU ; les latences d'interruption sont mesurées et consignées | ENF-01, ENF-08, ENF-21, EF-14 |
| M5 | Un programme ELF en EL0 affiche un message via `svc` ; un accès utilisateur à la mémoire noyau tue le processus sans faire tomber le noyau | EF-19 à EF-21, ENF-10 à ENF-13 |
| M6 | Un fichier de l'initrd est lu par un programme utilisateur ; création, écriture et relecture d'un fichier dans tmpfs | EF-22 |
| M7 | Le shell exécute une commande avec arguments, un tube entre deux programmes et une redirection vers un fichier | EF-23 à EF-25 |
| M8 | Le noyau charge le firmware du Cortex-M33 et échange des messages aller-retour avec lui ; le M33 tient sa boucle à fréquence fixe pendant que le noyau est chargé | EF-17, EF-18 |
| M9 | Une LED est pilotée par GPIO ; un signal PWM est vérifié à l'oscilloscope ou à l'analyseur logique ; une trame CAN-FD est émise et reçue en boucle | EF-16 |

| Jalon | Critères de recette | Exigences |
|---|---|---|
| M10 | Le noyau répond à un ping et échange des datagrammes UDP sous QEMU, puis sur la carte | EF-26 |

## 10. Risques et mitigations

Les deux risques principaux sont l'abandon en cours de route et l'indisponibilité de la carte ; le développement sur QEMU et la règle de portabilité couvrent le second, des jalons courts et démontrables le premier.

| Risque | Impact | Mitigation |
|---|---|---|
| Carte introuvable ou délai de livraison long | Portage matériel retardé | M0 à M3 entièrement sur QEMU ; cartes de repli (BeaglePlay, Raspberry Pi 4) ; règle ENF-01 |
| Perte de motivation ou de disponibilité (alternance, études) | Projet inachevé | Jalons courts et autonomes ; un noyau arrêté à M4 reste un projet complet et présentable |
| Bugs bas niveau difficiles à diagnostiquer | Blocages de plusieurs jours | GDB et GEF maîtrisés dès M0 ; décodage ESR / FAR dans `kpanic()` ; tests d'intégration systématiques |
| Timings non représentatifs sous QEMU | Fausse confiance sur le temps réel | Mesures sur carte obligatoires (ENF-08) |
| Complexité de la chaîne de boot de la carte (TF-A, U-Boot) | Blocage au portage | U-Boot charge le noyau tel quel ; la chaîne n'est modifiée qu'en phase sécurité |
| Absence de PAN sur le Cortex-A35 (ARMv8.0) | Mitigation incomplète | Émulation logicielle par bascule de TTBR0 ; PAN testé sous QEMU `-cpu max` |
| Erreurs à la frontière FFI | Corruptions silencieuses | Règles FFI-01 à FFI-04, bindings générés, revue des blocs `unsafe` |
| Dérive du périmètre (Wi-Fi, BLE, SMP, GUI) | Jalons jamais terminés | Périmètre figé en section 2 ; toute extension passe par un ADR après M10 |

| Risque | Impact | Mitigation |
|---|---|---|
| Accident électrique avec des moteurs | Matériel endommagé | Alimentation de puissance séparée, masses communes, arrêt d'urgence matériel indépendant du logiciel |

## 11. Évolutions envisagées et glossaire

Après M10, les évolutions possibles sont, par ordre de priorité : SMP sur les deux Cortex-A35, durcissement sécurité (KASLR) et exploitation contrôlée de vulnérabilités introduites volontairement, étude de TrustZone et du secure boot, gestion d'énergie avancée, puis Bluetooth LE et Wi-Fi.

| Terme | Définition |
|---|---|
| AArch64 | Mode d'exécution 64 bits de l'architecture ARMv8 |
| ADR | Architecture Decision Record : court document consignant une décision technique et sa justification |
| Device tree (FDT) | Description du matériel fournie par le bootloader au noyau, qui y découvre RAM et périphériques |
| EL0 / EL1 | Niveaux d'exception ARM : EL0 pour l'espace utilisateur, EL1 pour le noyau |
| FFI | Foreign Function Interface : mécanisme d'appel entre deux langages |
| Freestanding | Environnement d'exécution sans bibliothèque standard ni système d'exploitation sous-jacent |
| GIC | Generic Interrupt Controller : contrôleur d'interruptions standard des SoC ARM |
| Héritage de priorité | Un thread qui détient un mutex prend temporairement la priorité du thread le plus prioritaire qui l'attend |
| Higher-half | Noyau mappé dans la moitié haute de l'espace d'adressage virtuel |
| IPCC | Inter-Processor Communication Controller : mailbox matérielle entre les cœurs du STM32MP25 |

| Terme | Définition |
| --- | --- |
| `no_std` | Mode Rust sans bibliothèque standard, requis en environnement noyau |
| PAN / PXN / UXN | Protections ARM interdisant au noyau d'accéder à la mémoire utilisateur (PAN) ou d'exécuter du code d'un autre niveau (PXN, UXN) |
| TF-A | Trusted Firmware-A : firmware de référence ARM qui démarre le monde sécurisé avant le bootloader |
| U-Boot | Bootloader qui charge le noyau sur la carte |
| VFS | Virtual File System : couche d'abstraction commune à tous les systèmes de fichiers |
| W^X | Politique interdisant qu'une page mémoire soit à la fois inscriptible et exécutable |